Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопсности
Высшая школа программной инженерии

ЛАБОРАТОРНАЯ РАБОТА №5

по дисциплине «Теория автоматов и формальных языков»

Выполнил
студент гр. 5130904/30108 Ребдев П.А.

Проверил Тышкевич А.И.

Санкт-Петербург
2026

1. Формулировка задания

1. Задать контекстно-свободную грамматику, которая не является автоматной и не является S-грамматикой.
2. Проверить пригодность грамматики для разбора методом LL(1). При необходимости внести изменения.
3. Найти решающие правила (LOOKAHEAD) для каждой продукции и построить МП-автомат для нисходящего разбора.
4. Породить слово, выводимое не менее чем за 5 шагов, и привести протокол его распознавания МП-автоматом.
5. На основе протокола привести сам вывод слова и его дерево вывода.

2. Выбор грамматики

Для выполнения задания выбрана грамматика арифметических выражений с операциями сложения и умножения, приоритет операций задан обычным образом. Грамматика не является автоматной (содержит нетерминалы, порождающие рекурсивные структуры) и не является S-грамматикой, так как имеются продукции с пустой правой частью и правые части, начинающиеся с нетерминалов.

Грамматика G:

№	Продукция
0	S → E $
1	E → T A
2	A → + T A
3	A → ε
4	T → F B
5	B → * F B
6	B → ε
7	F → i
8	F → (E)

Терминальный алфавит: T = { + , * , i , (,) , $ }

Нетерминальный алфавит: N = { S , E , A , T , B , F }

Начальный символ: S

3. Проверка LL(1)-свойства

Для проверки пригодности грамматики к LL(1)-разбору необходимо вычислить множества FIRST и FOLLOW для всех нетерминалов, затем для каждой продукции определить решающее множество LOOKAHEAD и убедиться, что для каждого нетерминала эти множества попарно не пересекаются.

3.1. Вычисление FIRST

Выполняется итеративно до стабилизации.

Нетерминал	FIRST
S	{ i , (}
E	{ i , (}
A	{ + } (так как A → + T A, и A → ε – пусто)
T	{ i , (}
B	{ * }
F	{ i , (}

Детальный вывод:

- F → i даёт i, F → (E) даёт (→ FIRST(F) = {i,(}
- T → F B → FIRST(T) = FIRST(F) = {i,(}
- B → * F B → FIRST(B) = {*}
- A → + T A → FIRST(A) = {+}
- E → T A → FIRST(E) = FIRST(T) = {i,(}
- S → E $ → FIRST(S) = FIRST(E) = {i,(}

3.2. Вычисление FOLLOW

Строим с учётом всех вхождений нетерминалов в правые части.

Нетерминал	FOLLOW
S	{ $ } (начальный символ)
E	{ $,) } (из S → E $ и F → (E))
A	FOLLOW(E) = { $,) } (из E → T A, правее A ничего нет)
T	FIRST(A) ∪ FOLLOW(E) ∪ FOLLOW(A) = {+} ∪ {$,)} ∪ {$,)} = {+ , $,)}
B	FOLLOW(T) = {+ , $,)} (из T → F B)

F	FIRST(B) ∪ FOLLOW(T) ∪ FOLLOW(B) = {} ∪{+,$,)} ∪{+,$,)} = { , + , $,)}

3.3. Решающие множества LOOKAHEAD

Для каждой продукции:

- **0:** S → E $
  E $ не выводит ε → LOOKAHEAD = FIRST(E $) = FIRST(E) = { i , (}
- **1:** E → T A
  T A не выводит ε → LOOKAHEAD = FIRST(T A) = FIRST(T) = { i , (}
- **2:** A → + T A
  + T A начинается с терминала → LOOKAHEAD = { + }
- **3:** A → ε
  LOOKAHEAD = FOLLOW(A) = { $,) }
- **4:** T → F B
  F B не выводит ε → LOOKAHEAD = FIRST(F B) = FIRST(F) = { i , (}
- **5:** B → * F B
  * F B начинается с терминала → LOOKAHEAD = { * }
- **6:** B → ε
  LOOKAHEAD = FOLLOW(B) = { + , $,) }
- **7:** F → i
  LOOKAHEAD = { i }
- **8:** F → (E)
  LOOKAHEAD = { (}

3.4. Проверка непересечения

- Для нетерминала **A**:
  - A → + T A : {+}
  - A → ε : {$,)}
    Пересечения нет.
- Для нетерминала **B**:
  - B → * F B : {*}
  - B → ε : {+,$,)}
    Пересечения нет.
- Для нетерминала **F**:

- F → i : {i}
- F → (E) : {(}
  Пересечения нет.

Все условия выполнены → **грамматика является LL(1)**.

4. Построение МП-автомата и таблицы решающих правил

МП-автомат для нисходящего разбора имеет единственное состояние q. В стеке хранятся символы грамматики (терминалы и нетерминалы), дно обозначается ⊥. Управляющая таблица строится по решающим множествам: на пересечении нетерминала и входного символа записывается правая часть продукции.

Таблица решающих правил (LOOKAHEAD-таблица):

	+	*	i	(	)	$
S			E$	E$	S	
E			TA	TA	E	
A	+TA				ε	ε
T			FB	FB	T	
B	ε	*FB			ε	ε
F			i	(E)	F	

Правила работы автомата:

- Если на вершине стека **нетерминал** X, а текущий входной символ a, то по таблице находится правая часть α (или ε), и X заменяется на α (запись в стек справа налево). Позиция входа не меняется.
- Если на вершине стека **терминал** x и он совпадает с текущим входным символом a, то оба удаляются (стек и вход) – происходит продвижение.
- Если символы не совпадают или в таблице нет правила – ошибка.

Начальная конфигурация: стек содержит S⊥, входное слово оканчивается символом $.

5. Порождение слова и протокол распознавания

Выберем слово, выводимое не менее чем за 5 шагов:
i + i * i $ (в инфиксной записи i+i*i с маркером конца).

Ниже приведён пошаговый протокол работы МП-автомата. В каждой строке: номер итерации, содержимое стека (вершина слева) и оставшаяся часть входа.

(1): S,i+i*i$ // S,i → S->E$, i->i

(2): E$,i+i*i$ // E,i → E->TA, i->i

(3): TA$,i+i*i$ // T,i → T->FB, i->i

(4): FBA$,i+i*i$ // F,i → F->i, i->i

(5): iBA$,i+i*i$ // i,i → i->ε, i->ε

(6): BA$,+i*i$ // B,+ → B->ε, +->+

(7): A$,+i*i$ // A,+ → A->+TA, +->+

(8): +TA$,+i*i$ // +,+ → +->ε, +->ε

(9): TA$,i*i$ // T,i → T->FB, i->i

(10): FBA$,i*i$ // F,i → F->i, i->i

(11): iBA$,i*i$ // i,i → i->ε, i->ε

(12): BA$,*i$ // B,* → B->*FB, *->*

(13): *FBA$,*i$ // *,* → *->ε, *->ε

(14): FBA$,i$ // F,i → F->i, i->i

(15): iBA$,i$ // i,i → i->ε, i->ε

(16): BA$,$ // B,$ → B->ε, $->$

(17): A$,$ // A,$ → A->ε, $->$

(18): $,$ // $,$ → $->ε, $->ε

(19): , // Конец входа, стек пуст

Стек опустел одновременно с концом входа → **слово принадлежит языку**.

6. Вывод слова и его дерево вывода

6.1. Протокол вывода (с указанием применённых правил)

Вывод слова i + i * i $ в виде последовательности сентенциальных форм (номера шагов соответствуют итерациям, на которых происходила замена нетерминала):

(1) (2) (3) (4) (6) (7) (9) (10) (12)

S => E$ => TA$ => FBA$ => iBA$ => iA$ => i+TA$ => i+FBA$ => i+iBA$ =>

S->E$ E->TA T->FB F->i B->ε A->+TA T->FB F->i B->*FB

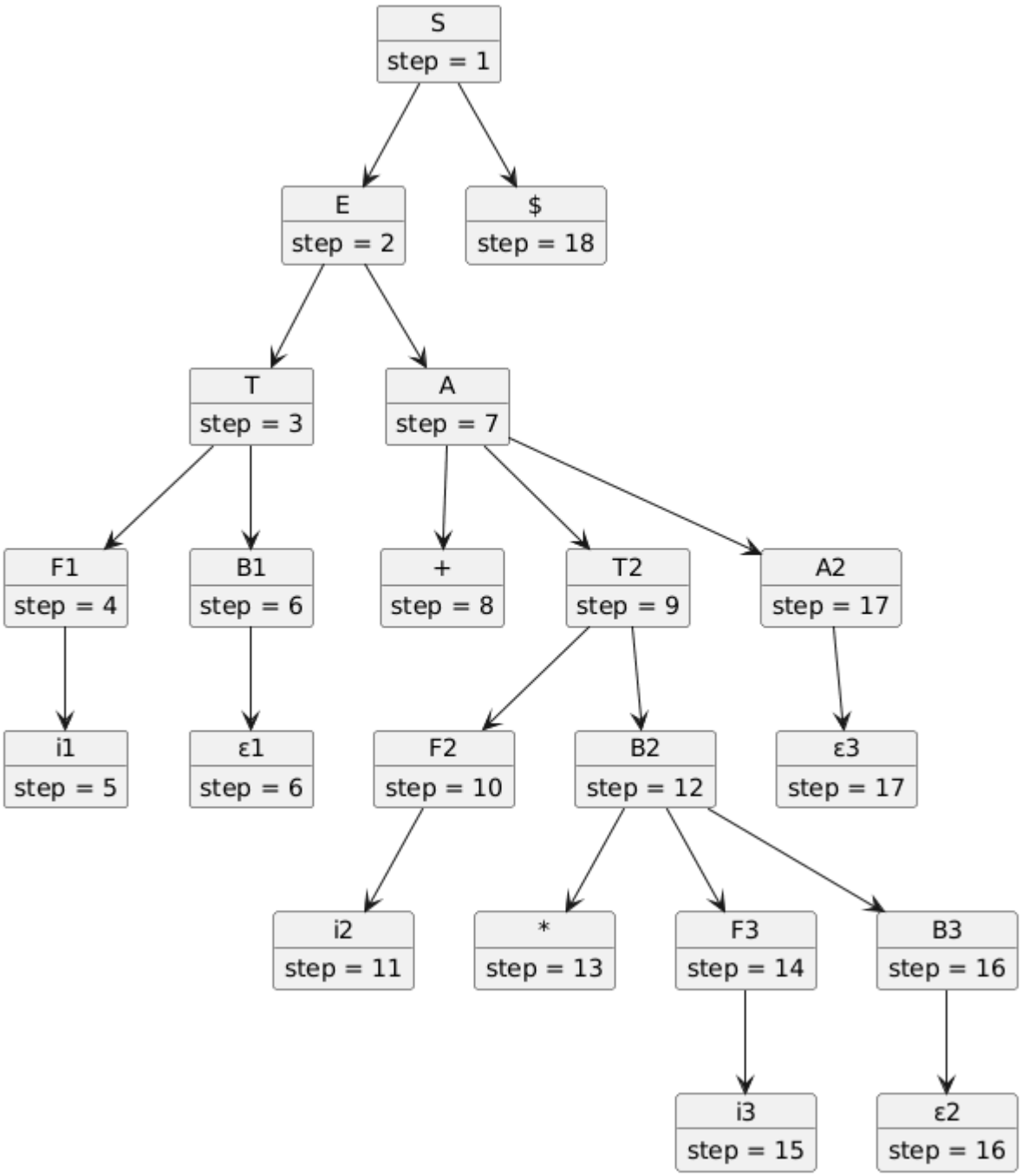
(14) (16) (17)

i+i*FBA$ => i+i*iBA$ => i+i*iA$ => i+i*i$

F->i B->ε A->ε

6.2. Дерево вывода (UML-диаграмма)

Ниже представлено дерево вывода в виде UML-диаграммы объектов (нотация PlantUML). Каждый объект соответствует узлу дерева, атрибут step указывает номер итерации, на которой узел был создан (для нетерминалов) или сопоставлен с входным символом (для терминалов). Пустые символы ε показаны как объекты с именем "ε".


S
step = 1
E
step = 2
$
step = 18
T
step = 3
A
step = 7
F1
step = 4
B1
step = 6
+
step = 8
T2
step = 9
A2
step = 17
i1
step = 5
ε1
step = 6
F2
step = 10
B2
step = 12
ε3
step = 17
i2
step = 11
*
step = 13
F3
step = 14
B3
step = 16
i3
step = 15
ε2
step = 16

7. Программная реализация

Для автоматизации всех вычислений и генерации протоколов разработана программа на C++. Она реализует:

- задание грамматики;
- вычисление FIRST, FOLLOW, LOOKAHEAD;
- проверку LL(1)-условия;
- построение таблицы решающих правил;
- моделирование работы МП-автомата для заданного входного слова;
- вывод пошагового протокола, последовательности вывода и дерева вывода.

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <stack>
#include <queue>
#include <algorithm>

struct Production
{
  char left;
  std::string right;
  int id;
};

bool isTerminal(char c)
{
  return c == '+' || c == '*' || c == 'i' || c == '(' || c == ')' || c == '$';
}

bool isNonTerminal(char c)
{
  return c == 'S' || c == 'E' || c == 'A' || c == 'T' || c == 'B' || c == 'F';
}

std::map< char, std::set< char > > computeFirst(
    const std::vector< Production > & prods,
    const std::set< char > & nonterms)
{
  std::map< char, std::set< char > > first;
  for (char nt : nonterms) first[nt] = std::set< char >();
  bool changed = true;
  while (changed)
  {
    changed = false;
    for (const auto & p : prods)
    {
      char A = p.left;
      std::string rhs = p.right;
      std::set< char > current;
```

```
      bool allNullable = true;
      size_t i = 0;
      while (i < rhs.size())
      {
        char sym = rhs[i];
        if (isTerminal(sym))
        {
          current.insert(sym);
          allNullable = false;
          break;
        }
        else
        {
          const auto & firstB = first[sym];
          for (char t : firstB)
            if (t != 0) current.insert(t);
          if (firstB.find(0) == firstB.end())
          {
            allNullable = false;
            break;
          }
          ++i;
        }
      }
      if (allNullable) current.insert(0);
      for (char t : current)
      {
        if (first[A].find(t) == first[A].end())
        {
          first[A].insert(t);
          changed = true;
        }
      }
    }
  }
  return first;
}

std::map< char, std::set< char > > computeFollow(
    const std::vector< Production > & prods,
    const std::set< char > & nonterms,
    const std::map< char, std::set< char > > & first,
    const std::map< char, bool > & nullable)
{
  std::map< char, std::set< char > > follow;
  for (char nt : nonterms) follow[nt] = std::set< char >();
  follow['S'].insert('$');
  bool changed = true;
  while (changed)
  {
    changed = false;
    for (const auto & p : prods)
    {
      char A = p.left;
      std::string rhs = p.right;
      for (size_t i = 0; i < rhs.size(); ++i)
      {
        char B = rhs[i];
        if (isNonTerminal(B))
        {
          std::set< char > firstBeta;
          bool allNullableBeta = true;
```

```
        size_t j = i + 1;
        while (j < rhs.size())
        {
          char sym = rhs[j];
          if (isTerminal(sym))
          {
            firstBeta.insert(sym);
            allNullableBeta = false;
            break;
          }
          else
          {
            const auto & firstB = first.at(sym);
            for (char t : firstB)
              if (t != 0) firstBeta.insert(t);
            if (firstB.find(0) == firstB.end())
            {
              allNullableBeta = false;
              break;
            }
            ++j;
          }
        }
        for (char t : firstBeta)
        {
          if (follow[B].find(t) == follow[B].end())
          {
            follow[B].insert(t);
            changed = true;
          }
        }
        if (allNullableBeta)
        {
          for (char t : follow[A])
          {
            if (follow[B].find(t) == follow[B].end())
            {
              follow[B].insert(t);
              changed = true;
            }
          }
        }
      }
    }
  }
  return follow;
}

bool derivesEpsilon(const std::string & alpha,
                    const std::map< char, bool > & nullable)
{
  for (char c : alpha)
  {
    if (isTerminal(c)) return false;
    if (!nullable.at(c)) return false;
  }
  return true;
}

std::map< int, std::set< char > > computeLookahead(
    const std::vector< Production > & prods,
```

```
    const std::map< char, std::set< char > > & first,
    const std::map< char, std::set< char > > & follow,
    const std::map< char, bool > & nullable)
{
  std::map< int, std::set< char > > la;
  for (const auto & p : prods)
  {
    std::string rhs = p.right;
    std::set< char > firstRhs;
    size_t i = 0;
    bool allNullable = true;
    while (i < rhs.size())
    {
      char sym = rhs[i];
      if (isTerminal(sym))
      {
        firstRhs.insert(sym);
        allNullable = false;
        break;
      }
      else
      {
        const auto & firstB = first.at(sym);
        for (char t : firstB)
          if (t != 0) firstRhs.insert(t);
        if (firstB.find(0) == firstB.end())
        {
          allNullable = false;
          break;
        }
        ++i;
      }
    }
    if (allNullable)
    {
      la[p.id] = firstRhs;
      for (char t : follow.at(p.left))
        la[p.id].insert(t);
    }
    else
    {
      la[p.id] = firstRhs;
    }
  }
  return la;
}

struct Node
{
  char symbol;
  int step;
  std::vector< Node * > children;
  Node(char s, int st) : symbol(s), step(st) {}
};

void printTree(Node * node, int depth, std::ostream & out)
{
  for (int i = 0; i < depth; ++i) out << "  ";
  if (node->symbol == 0)
    out << "ε";
  else
    out << node->symbol;
```

```
  if (node->step > 0) out << " " << node->step;
  out << "\n";
  for (Node * child : node->children)
    printTree(child, depth + 1, out);
}

void deleteTree(Node * node)
{
  for (Node * child : node->children)
    deleteTree(child);
  delete node;
}

int main()
{
  std::vector< Production > prods =
  {
    {'S', "E$", 0},
    {'E', "TA", 1},
    {'A', "+TA", 2},
    {'A', "", 3},
    {'T', "FB", 4},
    {'B', "*FB", 5},
    {'B', "", 6},
    {'F', "i", 7},
    {'F', "(E)", 8}
  };

  std::set< char > terminals = {'+', '*', 'i', '(', ')', '$'};
  std::set< char > nonterminals = {'S', 'E', 'A', 'T', 'B', 'F'};

  std::map< char, bool > nullable;
  for (char nt : nonterminals) nullable[nt] = false;
  bool changed = true;
  while (changed)
  {
    changed = false;
    for (const auto & p : prods)
    {
      if (p.right.empty())
      {
        if (!nullable[p.left])
        {
          nullable[p.left] = true;
          changed = true;
        }
        continue;
      }
      bool allNull = true;
      for (char c : p.right)
      {
        if (isTerminal(c)) { allNull = false; break; }
        if (!nullable[c]) { allNull = false; break; }
      }
      if (allNull && !nullable[p.left])
      {
        nullable[p.left] = true;
        changed = true;
      }
    }
  }
```

```
auto first = computeFirst(prods, nonterminals);
auto follow = computeFollow(prods, nonterminals, first, nullable);
auto lookahead = computeLookahead(prods, first, follow, nullable);

std::map< char, std::map< char, int > > table;
for (char nt : nonterminals)
  for (char t : terminals) table[nt][t] = -1;

bool ll1 = true;
for (const auto & p : prods)
{
  char A = p.left;
  for (char t : lookahead[p.id])
  {
    if (t == 0) continue;
    if (table[A][t] != -1)
    {
      std::cout << "Conflict for " << A << " on " << t
                << ": rules " << table[A][t] << " and " << p.id << "\n";
      ll1 = false;
      break;
    }
    table[A][t] = p.id;
  }
  if (!ll1) break;
}

if (!ll1)
{
  std::cout << "Grammar is not LL(1).\n";
  return 1;
}

std::cout << "Grammar is LL(1). LOOKAHEAD table:\n\n";
std::cout << "    | + | * | i | ( | ) | $\n";
std::cout << "----+---+---+---+---+---+---\n";
for (char nt : {'S', 'E', 'A', 'T', 'B', 'F'})
{
  std::cout << " " << nt << "  |";
  for (char t : {'+', '*', 'i', '(', ')', '$'})
  {
    int rid = table[nt][t];
    if (rid == -1)
      std::cout << "   |";
    else
      std::cout << " " << rid << " |";
  }
  std::cout << "\n";
}

std::cout << "\nEnter word to parse (must end with $): ";
std::string inputWord;
std::cin >> inputWord;

std::cout << "\nParsing: " << inputWord << "\n\n";

std::stack< std::pair< char, Node* > > st;
Node * root = new Node('S', 0);
st.push({'S', root});
std::string remaining = inputWord;
int step = 1;
std::vector< int > appliedRules;
```

```
std::vector< Node* > allNodes;
allNodes.push_back(root);

std::cout << "Parser trace:\n";
while (!(st.empty() && remaining.empty()))
{
  std::string stackStr;
  std::stack< std::pair< char, Node* > > temp = st;
  std::vector< char > rev;
  while (!temp.empty())
  {
    rev.push_back(temp.top().first);
    temp.pop();
  }
  for (auto it = rev.rbegin(); it != rev.rend(); ++it)
    stackStr.push_back(*it);
  std::cout << "(" << step << "):" << stackStr << "," << remaining << "\n";

  if (st.empty())
  {
    std::cout << "Error: empty stack, input not empty\n";
    break;
  }

  char topSym = st.top().first;
  Node * topNode = st.top().second;
  char nextInput = remaining.empty() ? 0 : remaining[0];

  if (isTerminal(topSym))
  {
    if (topSym == nextInput)
    {
      st.pop();
      remaining = remaining.substr(1);
      topNode->step = step;
    }
    else
    {
      std::cout << "Error: terminal " << topSym
                << " != input " << nextInput << "\n";
      break;
    }
  }
  else
  {
    int ruleId = table[topSym][nextInput];
    if (ruleId == -1)
    {
      std::cout << "Error: no rule for " << topSym
                << " on input " << nextInput << "\n";
      break;
    }
    appliedRules.push_back(ruleId);
    const Production & p = prods[ruleId];
    st.pop();
    std::string rhs = p.right;
    topNode->step = step;
    for (int i = rhs.size() - 1; i >= 0; --i)
    {
      char sym = rhs[i];
      Node * child = new Node(sym, 0);
      allNodes.push_back(child);
```

```
      topNode->children.insert(topNode->children.begin(), child);
      st.push({sym, child});
    }
    if (rhs.empty())
    {
      Node * eps = new Node(0, step);
      allNodes.push_back(eps);
      topNode->children.push_back(eps);
    }
  }
  ++step;
}

if (st.empty() && remaining.empty())
{
  std::cout << "(" << step << "):" << "," << "\n";
  std::cout << "Stack empty at end of input ===> Word accepted.\n";
}

std::cout << "\nDerivation steps (rules applied):\n";
int cnt = 0;
for (int rid : appliedRules)
{
  const Production & p = prods[rid];
  std::cout << "  (" << ++cnt << ") " << p.left << " -> "
            << (p.right.empty() ? "ε" : p.right) << "\n";
}

std::cout << "\nParse tree:\n";
printTree(root, 0, std::cout);

deleteTree(root);
return 0;
}
```

Результат работы программы:

```
Grammar is LL(1). LOOKAHEAD table:

    | + | * | i | ( | ) | $
----+---+---+---+---+---+---
 S  |   |   | 0 | 0 |   |   |
 E  |   |   | 1 | 1 |   |   |
 A  | 2 |   |   |   | 3 | 3 |
 T  |   |   | 4 | 4 |   |   |
 B  | 6 | 5 |   |   | 6 | 6 |
 F  |   |   | 7 | 8 |   |   |

Enter word to parse (must end with $):
Parsing: i+i*i$

Parser trace:
(1):S,i+i*i$
(2):$E,i+i*i$
(3):$AT,i+i*i$
(4):$ABF,i+i*i$
(5):$ABi,i+i*i$
(6):$AB,+i*i$
(7):$A,+i*i$
(8):$AT+,+i*i$
(9):$AT,i*i$
(10):$ABF,i*i$
(11):$ABi,i*i$
(12):$AB,*i$
(13):$ABF*,*i$
(14):$ABF,i$
(15):$ABi,i$
(16):$AB,$
(17):$A,$
(18):$,$
(19):,
Stack empty at end of input ===> Word accepted.
```

```
Derivation steps (rules applied):
  (1) S -> E$
  (2) E -> TA
  (3) T -> FB
  (4) F -> i
  (5) B -> ε
  (6) A -> +TA
  (7) T -> FB
  (8) F -> i
  (9) B -> *FB
  (10) F -> i
  (11) B -> ε
  (12) A -> ε

Parse tree:
S 1
  E 2
    T 3
      F 4
        i 5
      B 6
        ε 6
    A 7
      + 8
      T 9
        F 10
          i 11
        B 12
          * 13
          F 14
            i 15
          B 16
            ε 16
      A 17
        ε 17
  $ 18
```

8. Выводы

1. Выбрана контекстно-свободная грамматика для арифметических выражений, удовлетворяющая условиям (не автоматная, не S-грамматика).
2. Проведена проверка LL(1)-свойства: вычислены множества FIRST, FOLLOW и LOOKAHEAD для каждой продукции, подтверждено отсутствие конфликтов.
3. Построена управляющая таблица МП-автомата, реализующего нисходящий разбор.
4. Для слова i+i*i$ (длина вывода 12 шагов) получен детальный протокол работы автомата, последовательность применённых правил и дерево вывода.
5. Разработана программа на C++, автоматизирующая все вычисления и генерацию протоколов; она читает входное слово из консоли и выдаёт полный отчёт.
6. Полученные результаты демонстрируют работоспособность метода LL(1)-разбора для выбранной грамматики и могут быть использованы в дальнейшем при построении компиляторов.